

40-Minute Hands-On Tutorial

Build a “Cosmic Study Coach” with Cursor

What students will create

Students will build a colorful web app that asks:

- What subject are you studying?
- How difficult does it feel?
- How much time do you have?
- What is your current energy level?

The app will then generate a personalized “Study Quest” with activities, encouragement, and a progress mission.

The lesson follows the smart-feature structure shown in the reference lesson:

User input → Smart logic → Helpful output

For a 40-minute beginner activity, we will use a **rule-based smart feature with mock-AI presentation** rather than a live AI API. This matches the lesson’s recommendation to start with a simple, reliable feature before attempting a real cloud API.

Cursor’s Agent can inspect project files, create and edit code, and help test browser-based applications. ([Cursor](#))

Learning goals

By the end of the tutorial, students will be able to:

1. Create a web app with HTML, CSS, and JavaScript.
 2. use Cursor to generate and modify code.
 3. explain the input, logic, and output of a smart feature.
 4. test normal, empty, and unusual inputs.
 5. improve an app through follow-up prompts.
-

Final app

The completed app will include:

- A space-themed interface
- Subject input
- Difficulty selection
- Study-time selection

- Energy-level selection
- “Launch My Study Quest” button
- Animated “AI is planning...” message
- Personalized study plan
- Random motivational message
- Error handling
- “Try Another Quest” button

Before class

Students need:

- Cursor installed
- A modern web browser
- A folder where they can save the project

No API key, server, framework, or paid cloud service is required.

40-minute lesson schedule

Time	Activity
0–4 minutes	Introduce the challenge
4–8 minutes	Create the project
8–15 minutes	Ask Cursor to build the basic app
15–24 minutes	Add the smart feature
24–30 minutes	Run and test the app
30–36 minutes	Personalize the app
36–40 minutes	Final challenge and exit ticket

Part 1: Introduce the challenge

0–4 minutes

Tell students:

Today you are building a digital study coach for a student preparing for an important mission. Your coach will examine the student’s subject, difficulty, available time, and energy level. It will then create a personalized study quest.

Ask the class:

- What information should the app collect?
- What decision should the app make?
- What should the app produce?

Write this flow on the board:

```
INPUT
Subject + Difficulty + Time + Energy

↓

SMART LOGIC
JavaScript rules examine the choices

↓

OUTPUT
Personalized study quest
```

Explain that the app will feel intelligent because it responds differently to different combinations of inputs.

Part 2: Create the project

4–8 minutes

Step 1: Create a folder

Create a folder named:

```
cosmic-study-coach
```

Step 2: Open it in Cursor

In Cursor:

1. Select **Open Folder**.
2. Choose `cosmic-study-coach`.
3. Trust the folder when prompted.

Depending on the Cursor version, students may see an Agent panel, chat panel, or agent workspace. The labels can change, but the workflow remains the same: open the project, describe the task, review the proposed changes, and accept the files you want. Cursor's current desktop product is centered around agents that can create and change project code. ([Cursor](#))

Step 3: Open Cursor Agent

Open the Agent or chat panel.

Students should not begin with:

```
Make me an app.
```

That prompt is too vague.

Instead, explain the four parts of a useful coding prompt:

1. What are we building?
2. What technology should it use?
3. What features should it contain?
4. What files should Cursor create?

Part 3: Build the existing app

8–15 minutes

Before adding the smart feature, students will create the basic study-app interface.

Paste this into Cursor:

```
Create a beginner-friendly web app called "Cosmic Study Coach."
```

```
Use only:
```

- HTML
- CSS
- Vanilla JavaScript

```
Create these three files:
```

- index.html
- style.css
- script.js

```
For the first version, create only the user interface.
```

```
The page should include:
```

1. A title: "Cosmic Study Coach"
2. A subtitle: "Turn homework into a study mission."
3. A text input for the school subject
4. A difficulty dropdown with Easy, Medium, and Hard
5. A study-time dropdown with 15, 30, 45, and 60 minutes
6. An energy dropdown with Low, Medium, and High
7. A button labeled "Launch My Study Quest"
8. An empty result area below the button

```
Design requirements:
```

- Fun space theme
- Attractive to middle and high school students
- Responsive on laptops and phones
- Large readable text
- Rounded cards
- Subtle animations
- Do not use external frameworks or libraries

Link the CSS and JavaScript files correctly.
Add clear comments for beginning students.
Do not add the smart recommendation logic yet.

Step 4: Review Cursor's plan

Before accepting changes, students should check:

- Did Cursor create exactly three files?
- Is the CSS linked in `index.html`?
- Is `script.js` linked near the bottom of the HTML?
- Are all four inputs present?
- Is there a result section?

Tell students:

Cursor is your coding partner, not your boss. You are responsible for checking its work.

Step 5: Accept the changes

Accept or apply Cursor's proposed edits.

The folder should now look like this:

```
cosmic-study-coach/  
├─ index.html  
├─ style.css  
└─ script.js
```

Part 4: Add the smart feature

15–24 minutes

Now students will add the intelligent behavior.

Step 6: Define the smart-feature rules

Explain that the app will use several decision rules.

Examples:

```
IF difficulty is Hard
THEN include a short concept review.

IF energy is Low
THEN use short work sessions and more breaks.

IF study time is 15 minutes
THEN generate a compact mission.

IF study time is 60 minutes
THEN generate a longer mission with review and practice.
```

This is a Level 1 rule-based feature, while the loading message and polished output give it some Level 2 mock-AI presentation. The reference lesson describes rule-based features as predictable and appropriate for beginner projects, and mock AI as useful for prototypes that resemble an AI experience.

Step 7: Ask Cursor to implement the logic

Paste this follow-up prompt into the same Cursor conversation:

```
Now add the smart feature to the existing Cosmic Study Coach app.

Do not rebuild the project from scratch. Modify the existing files.

When the user clicks "Launch My Study Quest":

1. Validate the subject input.
2. If the subject is blank, show a friendly error message.
3. Show an animated message saying:
   "Scanning the learning galaxy..."
4. Wait approximately 1.2 seconds.
5. Generate a personalized study quest based on:
   - subject
   - difficulty
   - available time
   - energy level

Use understandable JavaScript if/else logic.

Rules:

DIFFICULTY
- Easy: focus on confidence, review, and one small challenge
- Medium: combine review and practice
- Hard: begin with concept review, then guided practice, then a challenge

ENERGY
- Low: use short 5-minute work blocks, include a break, and use encouraging
```

language

- Medium: use balanced work blocks
- High: include a bonus challenge

TIME

- 15 minutes: create 2 short tasks
- 30 minutes: create 3 tasks
- 45 minutes: create 4 tasks
- 60 minutes: create 5 tasks including a final review

The output should include:

- A mission title using the subject
- A numbered study plan
- A personalized explanation
- One randomly selected motivational message
- A “Try Another Quest” button

Add at least six motivational messages.

Make the result appear as an attractive animated card.

Keep the JavaScript readable for middle and high school beginners.

Add comments explaining:

- reading user input
- validation
- decision rules
- building the output
- displaying the result

Do not use an external AI API.

Do not collect or save personal information.

Step 8: Inspect the generated JavaScript

Ask students to find these ideas in `script.js`:

```
document.getElementById(...)
```

This finds elements on the page.

```
addEventListener("click", ...)
```

This runs code when the button is clicked.

```
if (...) {  
  // decision
```

```
}
```

This applies smart rules.

```
setTimeout(...)
```

This briefly displays the “thinking” animation.

```
result.innerHTML = ...
```

This displays the generated plan.

Students do not have to understand every line yet. They should be able to identify:

- Where input is collected
- Where decisions happen
- Where output is displayed

Part 5: Run the app

24–27 minutes

Students can open `index.html` directly in the browser.

For the simplest method:

1. Find `index.html` in the Cursor file panel.
2. Right-click it.
3. Choose **Reveal in File Explorer** or **Reveal in Finder**.
4. Double-click `index.html`.

A browser should open the app.

An optional classroom setup is to install a local preview extension such as Live Server before the lesson, but it is not required for this project.

Part 6: Test the smart feature

27–30 minutes

The reference lesson recommends testing normal input, empty input, unusual input, and repeated clicks.

Students should complete this test table.

Test	Input	Expected result
Normal	Biology, Medium, 30 min, Medium energy	Three useful study tasks
Short mission	Algebra, Hard, 15 min, Low energy	Two short tasks and encouragement
Long mission	History, Easy, 60 min, High energy	Five tasks and a bonus challenge
Empty subject	Leave subject blank	Friendly error message
Strange input	123!!!	App should not crash
Repeat click	Click button several times	New result appears without breaking

Bug-fixing prompt

When something does not work, students should paste this into Cursor:

```
The app is not behaving correctly.

Please inspect index.html, style.css, and script.js.

Check:
1. Whether all element IDs match
2. Whether script.js is linked correctly
3. Whether the click event runs
4. Whether blank input is handled
5. Whether repeated button clicks work
6. Whether there are JavaScript console errors

Fix only the problems you find.
Do not redesign the entire app.
Explain the cause of each problem in simple language.
```

This is an important vibe-coding habit:

```
Build → Run → Observe → Describe → Fix → Test again
```

Part 7: Personalize the app

30–36 minutes

Students choose one enhancement.

They should make only one change at a time.

Option A: Add a subject emoji

Improve the result card by displaying an emoji based on the subject.

Examples:

- Math: 📊
- Science or Biology: 🔬
- History: 🏛️
- English or Writing: 📝
- Computer Science or Coding: 💻
- Art: 🎨
- Other subjects: 🚀

Keep all existing behavior.

Option B: Add a mission score

Add a “Mission Power Score” from 1 to 100.

Calculate it using:

- difficulty
- study time
- energy level

Show the score with a progress bar in the result card.

Explain the score as a fun game value, not as a judgment of the student.
Keep the calculation simple and readable.

Option C: Add celebration effects

Add a short celebration effect when a study quest is generated.

Use small star or confetti-like elements created with CSS and JavaScript.
The effect should last no more than two seconds.
Do not use an external library.
Do not make the page difficult to read.

Option D: Add a surprise challenge

Add one optional surprise challenge to each plan.

Create at least eight possible challenges, such as:

- Explain one idea to an imaginary alien
- Draw a concept as a comic
- Create a three-question quiz
- Teach the topic to a stuffed animal

- Write the hardest idea in one sentence

Randomly select one challenge.

Do not remove the existing study plan.

Option E: Change the theme

Change the Cosmic Study Coach into an underwater exploration theme.

Update:

- title
- background
- icons
- button text
- loading message
- result-card wording

Keep all functionality and input fields unchanged.

Do not rewrite the JavaScript logic unless text changes are necessary.

Part 8: Final two-minute challenge

36–38 minutes

Students must improve one sentence in the interface without asking Cursor to rewrite the whole project.

Example prompt:

Change only the subtitle to something more exciting for teenagers.

Give me three subtitle choices first.

Do not edit any files until I choose one.

This teaches an important workflow:

1. Ask Cursor for options.
2. Make a human decision.
3. Ask Cursor to implement the selected option.

The student remains the designer.

Part 9: Exit ticket

38–40 minutes

Each student answers:

1. What is the input?

Example:

The user enters a subject and chooses difficulty, time, and energy.

2. What is the smart logic?

Example:

JavaScript uses if/else rules to select the number and type of study activities.

3. What is the output?

Example:

The app displays a personalized study quest and motivational message.

4. How does the feature help the user?

Example:

It turns a vague goal such as “study biology” into a short, actionable plan.

5. Is this a real AI API?

Expected answer:

No. It is a rule-based smart feature with a mock-AI experience.

Complete fallback code

Use this section only when Cursor produces incomplete or broken code. Students should first try prompting, reviewing, and debugging.

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">

  <!-- Makes the page resize correctly on phones and tablets. -->
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Cosmic Study Coach</title>
```

```
<!-- Connects this page to our CSS file. -->
<link rel="stylesheet" href="style.css">
</head>

<body>
  <main class="app-container">
    <section class="hero">
      <div class="planet"> 🌍 </div>
      <p class="eyebrow">STUDENT MISSION CONTROL</p>
      <h1>Cosmic Study Coach</h1>
      <p class="subtitle">Turn homework into a study mission.</p>
    </section>

    <section class="input-card">
      <div class="form-group">
        <label for="subject">What are you studying?</label>
        <input
          id="subject"
          type="text"
          placeholder="Example: Biology"
          maxlength="50"
        >
      </div>

      <div class="form-grid">
        <div class="form-group">
          <label for="difficulty">Difficulty</label>
          <select id="difficulty">
            <option value="easy">Easy</option>
            <option value="medium" selected>Medium</option>
            <option value="hard">Hard</option>
          </select>
        </div>

        <div class="form-group">
          <label for="studyTime">Available time</label>
          <select id="studyTime">
            <option value="15">15 minutes</option>
            <option value="30" selected>30 minutes</option>
            <option value="45">45 minutes</option>
            <option value="60">60 minutes</option>
          </select>
        </div>

        <div class="form-group">
          <label for="energy">Energy level</label>
          <select id="energy">
            <option value="low">Low</option>
            <option value="medium" selected>Medium</option>
            <option value="high">High</option>
          </select>
        </div>
      </div>
    </section>
  </main>
</body>
```

```

        <button id="launchButton" type="button">
            🚀 Launch My Study Quest
        </button>

        <p id="errorMessage" class="error-message" aria-live="polite"></p>
    </section>

    <!-- JavaScript will place the loading message and study quest here. -->
    <section id="result" class="result-area" aria-live="polite"></section>
</main>

<!-- Connects the JavaScript after the HTML has loaded. -->
<script src="script.js"></script>
</body>
</html>

```

style.css

```

/* Makes element sizes easier to calculate. */
* {
    box-sizing: border-box;
}

:root {
    --background-dark: #11152f;
    --background-light: #292d64;
    --card: rgba(255, 255, 255, 0.12);
    --card-border: rgba(255, 255, 255, 0.2);
    --text: #f7f7ff;
    --muted: #c9cbcd;
    --accent: #ff8c69;
    --accent-light: #ffdd16;
    --error: #ffb4b4;
}

body {
    min-height: 100vh;
    margin: 0;
    padding: 30px 16px;
    font-family: Arial, Helvetica, sans-serif;
    color: var(--text);
    background:
        radial-gradient(circle at 20% 20%, #6b4eff 0, transparent 25%),
        radial-gradient(circle at 80% 10%, #e85aad 0, transparent 22%),
        linear-gradient(145deg, var(--background-dark), var(--background-light));
}

body::before {
    content: "";
    position: fixed;

```

```
    inset: 0;
    pointer-events: none;
    opacity: 0.25;
    background-image:
      radial-gradient(circle, white 1px, transparent 1px);
    background-size: 35px 35px;
  }

.app-container {
  position: relative;
  width: min(760px, 100%);
  margin: 0 auto;
}

.hero {
  margin-bottom: 22px;
  text-align: center;
}

.planet {
  display: inline-block;
  font-size: 58px;
  animation: float 3s ease-in-out infinite;
}

.eyebrow {
  margin: 8px 0;
  color: var(--accent-light);
  font-size: 13px;
  font-weight: bold;
  letter-spacing: 2px;
}

h1 {
  margin: 6px 0;
  font-size: clamp(34px, 7vw, 58px);
}

.subtitle {
  margin: 6px 0 0;
  color: var(--muted);
  font-size: 18px;
}

.input-card,
.quest-card,
.loading-card {
  border: 1px solid var(--card-border);
  border-radius: 24px;
  background: var(--card);
  box-shadow: 0 18px 45px rgba(0, 0, 0, 0.25);
  backdrop-filter: blur(14px);
}
```

```
.input-card {
  padding: 24px;
}

.form-grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 14px;
}

.form-group {
  margin-bottom: 18px;
}

label {
  display: block;
  margin-bottom: 8px;
  font-weight: bold;
}

input,
select {
  width: 100%;
  border: 2px solid transparent;
  border-radius: 12px;
  padding: 13px;
  color: #202344;
  background: #ffffff;
  font-size: 16px;
}

input:focus,
select:focus {
  border-color: var(--accent-light);
  outline: none;
}

button {
  width: 100%;
  border: 0;
  border-radius: 14px;
  padding: 15px 18px;
  color: #22223b;
  background: linear-gradient(135deg, var(--accent-light), var(--accent));
  font-size: 17px;
  font-weight: bold;
  cursor: pointer;
  transition: transform 0.2s ease, box-shadow 0.2s ease;
}

button:hover {
  transform: translateY(-2px);
  box-shadow: 0 10px 25px rgba(255, 140, 105, 0.35);
}
```

```
button:active {
  transform: translateY(0);
}

.error-message {
  min-height: 20px;
  margin: 12px 0 0;
  color: var(--error);
  font-weight: bold;
  text-align: center;
}

.result-area {
  margin-top: 20px;
}

.loading-card {
  padding: 30px;
  text-align: center;
}

.loading-stars {
  margin-bottom: 12px;
  font-size: 35px;
  animation: pulse 0.8s ease-in-out infinite alternate;
}

.quest-card {
  padding: 26px;
  animation: appear 0.45s ease-out;
}

.quest-label {
  margin: 0;
  color: var(--accent-light);
  font-size: 13px;
  font-weight: bold;
  letter-spacing: 1.5px;
}

.quest-card h2 {
  margin: 8px 0 12px;
  font-size: 30px;
}

.quest-summary {
  color: var(--muted);
  line-height: 1.6;
}

.quest-list {
  padding-left: 22px;
}
```

```
.quest-list li {
  margin: 13px 0;
  padding-left: 5px;
  line-height: 1.5;
}

.motivation {
  margin: 22px 0;
  border-left: 5px solid var(--accent-light);
  border-radius: 10px;
  padding: 14px;
  background: rgba(255, 209, 102, 0.13);
  font-weight: bold;
}

.secondary-button {
  color: var(--text);
  background: rgba(255, 255, 255, 0.13);
  border: 1px solid var(--card-border);
}

@keyframes float {
  0%,
  100% {
    transform: translateY(0) rotate(-4deg);
  }

  50% {
    transform: translateY(-10px) rotate(4deg);
  }
}

@keyframes pulse {
  from {
    transform: scale(0.9);
    opacity: 0.6;
  }

  to {
    transform: scale(1.1);
    opacity: 1;
  }
}

@keyframes appear {
  from {
    transform: translateY(15px);
    opacity: 0;
  }

  to {
    transform: translateY(0);
    opacity: 1;
  }
}
```

```

    }
  }

  @media (max-width: 650px) {
    .form-grid {
      grid-template-columns: 1fr;
      gap: 0;
    }

    .input-card,
    .quest-card {
      padding: 20px;
    }
  }
}

```

script.js

```

// Find the important elements in the HTML.
const subjectInput = document.getElementById("subject");
const difficultySelect = document.getElementById("difficulty");
const studyTimeSelect = document.getElementById("studyTime");
const energySelect = document.getElementById("energy");
const launchButton = document.getElementById("launchButton");
const resultArea = document.getElementById("result");
const errorMessage = document.getElementById("errorMessage");

// Messages are selected randomly to make the app feel less repetitive.
const motivationalMessages = [
  "Small steps can move an entire spaceship.",
  "You do not need to know everything—you only need to begin.",
  "Every question you answer adds experience points.",
  "Confusion is often the first stage of learning.",
  "Your future self will be glad you completed this mission.",
  "Progress beats perfection in every galaxy.",
  "One focused study session can change your whole day.",
  "The hardest mission becomes easier after the first step."
];

// This function chooses one random item from an array.
function getRandomItem(items) {
  const randomIndex = Math.floor(Math.random() * items.length);
  return items[randomIndex];
}

// This function creates the tasks for the study plan.
function buildStudyTasks(subject, difficulty, studyTime, energy) {
  const tasks = [];

  // Decide how long each work block should be.
  let blockLength = 8;

```

```
if (energy === "low") {
  blockLength = 5;
} else if (energy === "high") {
  blockLength = 10;
}

// Add tasks based on difficulty.
if (difficulty === "easy") {
  tasks.push(
    `Review your notes about ${subject} for ${blockLength} minutes.`
  );

  tasks.push(
    `Complete one confidence-building ${subject} example without rushing.`
  );
} else if (difficulty === "medium") {
  tasks.push(
    `Review the three most important ideas in ${subject} for
${blockLength} minutes.`
  );

  tasks.push(
    `Complete two practice questions and check your reasoning.`
  );
} else {
  tasks.push(
    `Choose the most confusing ${subject} idea and review it for
${blockLength} minutes.`
  );

  tasks.push(
    `Study one worked example and explain each step in your own words.`
  );

  tasks.push(
    `Attempt one challenging question without looking at the answer
first.`
  );
}

// Low-energy students receive a planned reset.
if (energy === "low") {
  tasks.push(
    "Take a two-minute recharge break: stretch, breathe, and drink water."
  );
}

// High-energy students receive an optional bonus challenge.
if (energy === "high") {
  tasks.push(
    `Bonus mission: teach one ${subject} idea aloud as if you were
teaching an alien.`
  );
}
```

```
// Add more tasks when the student has more time.
if (studyTime >= 30) {
  tasks.push(
    `Create three quick questions about ${subject}, then answer them.`
  );
}

if (studyTime >= 45) {
  tasks.push(
    "Mark anything that still feels confusing and write one question to
ask later."
  );
}

if (studyTime >= 60) {
  tasks.push(
    `Finish with a five-minute review: summarize today's ${subject}
learning in three sentences.`
  );
}

// Choose the required number of tasks based on available time.
let requiredTaskCount = 2;

if (studyTime === 30) {
  requiredTaskCount = 3;
} else if (studyTime === 45) {
  requiredTaskCount = 4;
} else if (studyTime === 60) {
  requiredTaskCount = 5;
}

// Remove extra tasks so the plan matches the selected study time.
return tasks.slice(0, requiredTaskCount);
}

// This function creates an explanation based on the student's selections.
function buildExplanation(difficulty, studyTime, energy) {
  let energyText = "balanced work blocks";

  if (energy === "low") {
    energyText = "short tasks and a recharge break";
  } else if (energy === "high") {
    energyText = "focused tasks and an extra challenge";
  }

  return `This ${studyTime}-minute mission uses ${energyText}.
  It is designed for a ${difficulty} topic so the plan matches the challenge
level you selected.`;
}

// This function displays the final study quest.
function displayQuest(subject, difficulty, studyTime, energy) {
```

```
const tasks = buildStudyTasks(
  subject,
  difficulty,
  studyTime,
  energy
);

const explanation = buildExplanation(
  difficulty,
  studyTime,
  energy
);

const motivation = getRandomItem(motivationalMessages);

// Convert every task into an HTML list item.
const taskListHtml = tasks
  .map((task) => `<li>${task}</li>`)
  .join("");

// Build and display the complete result card.
resultArea.innerHTML = `
  <article class="quest-card">
    <p class="quest-label">MISSION READY</p>

    <h2>${subject} Exploration Quest</h2>

    <p class="quest-summary">${explanation}</p>

    <ol class="quest-list">
      ${taskListHtml}
    </ol>

    <div class="motivation">
      ★ ${motivation}
    </div>

    <button
      id="tryAgainButton"
      class="secondary-button"
      type="button"
    >
      🔄 Try Another Quest
    </button>
  </article>
`;

// The new button was just created, so we find it after displaying the card.
const tryAgainButton = document.getElementById("tryAgainButton");

tryAgainButton.addEventListener("click", generateQuest);
}
```

// This function reads input, validates it, and begins the smart process.

```
function generateQuest() {
  // Read the current user selections.
  const subject = subjectInput.value.trim();
  const difficulty = difficultySelect.value;
  const studyTime = Number(studyTimeSelect.value);
  const energy = energySelect.value;

  // Remove any older error message.
  errorMessage.textContent = "";

  // Stop if the subject is blank.
  if (subject === "") {
    errorMessage.textContent =
      "Please enter a subject before launching your mission.";

    resultArea.innerHTML = "";
    subjectInput.focus();
    return;
  }

  // Display a short mock-AI loading experience.
  resultArea.innerHTML = `
    <div class="loading-card">
      <div class="loading-stars">✨ 🚀 ✨</div>
      <strong>Scanning the learning galaxy...</strong>
    </div>
  `;

  launchButton.disabled = true;
  launchButton.textContent = "Planning Your Mission...";

  // Wait briefly before displaying the plan.
  setTimeout(() => {
    displayQuest(
      subject,
      difficulty,
      studyTime,
      energy
    );

    launchButton.disabled = false;
    launchButton.textContent = "🚀 Launch My Study Quest";
  }, 1200);
}

// Run the smart feature whenever the main button is clicked.
launchButton.addEventListener("click", generateQuest);

// Also allow the Enter key to launch a quest from the subject box.
subjectInput.addEventListener("keydown", (event) => {
  if (event.key === "Enter") {
    generateQuest();
  }
});
```

```
}  
});
```

Teacher discussion: Is this “real AI”?

This project should be described accurately.

The web app itself does not send information to a large language model. Its intelligence comes from:

- User-specific inputs
- Decision rules
- Randomized responses
- Personalized output
- A polished AI-like interaction

Cursor is the AI coding assistant used to help students create the app. The finished app is a **rule-based smart app with a mock-AI user experience**.

This is a strong choice for a 40-minute classroom activity because it is:

- Fast to build
- Easy to test
- Free to run
- Safe for student use
- Predictable during demonstrations
- Easy to understand and modify

The reference lesson similarly emphasizes that “smart” is about creating a useful user experience, not merely adding complex code or a live AI API.

Optional future upgrade

In a later lesson, students could replace the rule-based generator with a real AI API. That version should use:

```
Browser → Your secure backend → AI provider
```

Students should **not** place a secret API key directly in `script.js`, because browser code can be viewed by users. A real API lesson would also need:

- A backend server
- Environment variables
- Usage limits
- Input filtering
- Error handling

- Privacy rules
- Teacher-managed API access

For this lesson, the simpler implementation is the more reliable and educational choice.